

An LLM Wiki System for High-Consequence Domain Knowledge Organization: Initial Feasibility for Space Operations and Nuclear Engineering

Ju-Hyeok Park^{1,2}, Hogeon Seo^{3,4,*}

¹Intern, Korea Atomic Energy Research Institute, Daejeon 34057, Republic of Korea

²Department of Electronic Engineering, Chungbuk National University, Cheongju 28644, Republic of Korea

³Senior Researcher, Korea Atomic Energy Research Institute, Daejeon 34057, Republic of Korea

⁴Associate Professor, University of Science and Technology, Daejeon 34113, Republic of Korea

*Corresponding author

Abstract

Large language model applications in high-consequence domains such as space operations and nuclear engineering motivate knowledge that is organized and reliably retrieved during interaction, not only at query time. We present an LLM wiki system for knowledge organization in such domains, evaluated as an initial feasibility study on a general technical corpus. The model both writes the wiki, as schema-bound notes from ordinary sessions, and reads it back. Documents are embedded incrementally and connected through ambient context injection. On each turn a hook selects relevant nodes and inserts them before generation, with no manual search. The contribution is this working integration and its feasibility measurement, not a new retrieval algorithm or a claim of component superiority. Judged blind by two cross-family models, ambient injection retrieves the answer-bearing node substantially more often than a hybrid baseline, most strongly where question and source wording differ, with higher answer correctness and fewer refusals. A small check on real space and nuclear abstracts shows the same direction on raw counts, not a replicated effect size. We also report negative findings: graph-level interventions do not help here, and supplying the answer-bearing node directly still leaves substantial hallucination, motivating independent verification as future work.

Keywords: knowledge organization; context injection; retrieval grounding; space operations; nuclear domain

1. Introduction

Large language model applications increasingly depend on private, domain-specific knowledge bases. A useful knowledge layer must retrieve the right source *during* interaction, not merely store documents or expose a manual search interface. Retrieval-augmented generation [1] supplies external documents at query time, but treats retrieval as an explicit query step separate from the interaction, and, as hallucination surveys note [3], it does not remove ungrounded generation. A usable system must also organize what accumulates: keep structure, identify repeated content, expose relations, and detect underdeveloped areas.

These needs are acute in high-consequence domains. In nuclear engineering, terminology is specialized and the relations among concepts, systems, procedures, and evidence must stay traceable. Space operations add a further constraint: mission design and long-duration crewed operations depend on large, evolving documentation while the link to ground is intermittent and bandwidth-limited, so an assistant must draw on a locally maintained knowledge base rather than open search [4], and an ungrounded answer is safety-relevant. Both domains pose the same problem: knowledge that must be organized, kept current, and grounded at the point of use.

We present an LLM wiki system for this problem, built on seven functions over an incremental embedding layer (Section 2), whose central function injects relevant nodes automatically on each user turn. The contribution is the implemented integration and its feasibility evaluation, not a new retriever and not an isolated measurement of the ambient trigger, which a single-turn benchmark cannot identify (Section 4). We evaluate on a general technical corpus, with a consistency check on real space and nuclear text; we do not measure domain safety outcomes.

2. System description

The wiki is not an external corpus that the system merely reads: the model writes it. The system therefore spans construction and use through seven functions (F0–F6), shown in Figure 1. F0 produces the wiki, and F1 builds the index that the others read. F2–F5 are human-facing: they run offline over

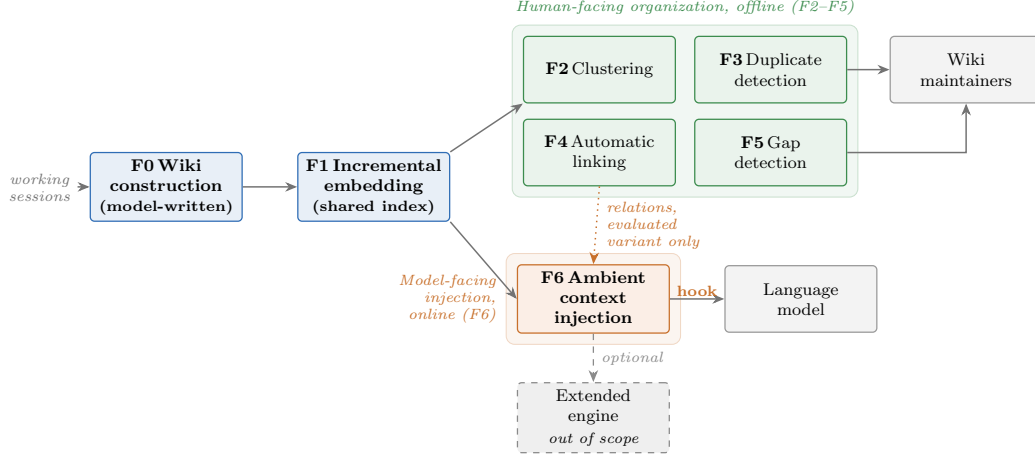


Fig. 1. LLM wiki architecture. F0 writes the wiki from ordinary working sessions; F1 embeds those notes into a shared index feeding two independent branches. Above, F2–F5 (green) read the index offline and report to the maintainers. Below, F6 (orange) reads the same index online and injects the selected nodes through a hook on user input; F6 does not consume F2, F3, or F5, and F4’s relations attach to already-selected nodes only in the evaluated variant C3 of Section 3 (dotted). The extended engine (hatched) is out of scope.

that index and report redundancy, relations, and gaps to the people who maintain the wiki. F6 is model-facing: it runs online during a turn, queries the same index, and places the retrieved text into the model’s context with no user action. Together the two branches let curated knowledge reach the model without anyone searching for it. This differs from a retrieval-augmented pipeline [1] in two ways: the corpus is model-written rather than supplied externally, and F6 needs no query. It also differs from an ontology: F0’s types fix only a note’s fields, and F4’s relations are proposed after the fact from embedding similarity rather than declared in advance.

F0: Wiki construction. At the end of a working session the model writes what it produced as a schema-bound markdown note of one of four types (precedent, decision, research, system), each with a fixed field set so notes stay comparable. The wiki therefore grows from ordinary use rather than from separate authoring, and the corpus that F1 indexes is model-written and human-curated.

F1: Incremental embedding. Each wiki document is embedded into a 1,024-dimensional BGE-M3 [2] vector and assigned to one of eight categories; we call each embedded document a *node*. Embedding is incremental, so newly added or edited documents are represented without rebuilding the entire corpus, which keeps the knowledge layer current as the wiki grows.

F2: Clustering. K-means over the embedding space groups nodes into topical clusters, giving maintainers a navigable view of what the corpus covers.

F3: Duplicate detection. Node pairs at or above 0.92 cosine (near-identity) are flagged as near-duplicates, surfacing redundant or superseded content for maintenance.

F4: Automatic linking. Node pairs at or above 0.75 cosine (topical relatedness) are proposed as relations, exposing broader connections than explicit links alone. They support navigation, gap detection (F5), and maintenance; consistent with the null result in Section 3.4 we do not claim they improve retrieval.

F5: Gap detection. A nearest-neighbor search marks regions where expected relations or coverage appear weak, for the maintainers to write up.

F6: Ambient context injection. This is the central, evaluated function, and it runs as four steps on every user turn. (i) A pre-processing hook fires on the user’s input before the model sees it, so no explicit search command is needed. (ii) The hook embeds the prompt and scores it against all node vectors by cosine similarity. (iii) Among nodes clearing a minimum-similarity floor of 0.55, the hook keeps the three highest-scoring, so a turn with no sufficiently related node receives no injected context; the floor is lower than the F3 and F4 thresholds because it filters noise rather than asserting near-identity or relatedness. (iv) The bodies of the kept nodes are prepended to the prompt and generation proceeds, subject to a 7.0-second budget so that a slow or unavailable embedding service degrades to the unmodified prompt instead of blocking the turn.

Because step (iii) withholds weak context rather than always filling three slots, part of the refusal

Table 1. Retrieval quality on 56 answerable questions. Recall@3, the operational hit rate, denotes whether the gold node is among the injected context. C0 is omitted because it retrieves nothing by construction. C3 selects the same three nodes as C2 on 53 of the 56 questions, the null reported in Section 3.4.

Metric	Hybrid (C1)	Ambient (C2)	+Graph (C3)
Recall@1	34%	55%	54%
Recall@3 (hit rate)	55%	79%	79%
Hit rate, paraphrase	39%	75%	75%
Hit rate, compositional	64%	79%	79%

reduction in Section 3.3 may reflect the absence of injected noise rather than better grounding. The hook also exposes graph-derived and reuse-based variants, evaluated as C3, and a lexical fallback when the embedding service is unavailable.

3. Results

3.1. Setup

We evaluate on a general technical wiki of roughly 6,000 notes across eight categories, maintained by a research group as a working record of research notes, design decisions, experiments, and incidents. The corpus is not nuclear-domain material, so the experiment measures implementation feasibility rather than nuclear-domain performance. We construct 56 answerable questions (14 extractive, 28 paraphrase, 14 compositional): extractive questions have an answer stated in one node, paraphrase questions express the source answer with different wording, and compositional questions require two nodes. Each has an annotated answer-bearing gold node and a gold answer span. We additionally include 16 unanswerable questions whose correct behavior is refusal; percentages below are based on the 56 answerable questions unless stated otherwise.

Answers are generated by a single frontier language model and graded by two frontier models from different families under blind conditions: the retrieval condition is hidden from the judge and grading is anchored to the gold span rather than a reference answer. The two judges agree on the correctness label in 94% of cases (raw agreement); we report agreement rather than a chance-corrected coefficient. Automated judging is a scalable instrument for a feasibility study, not a substitute for expert judgment in a high-consequence domain, and human adjudication remains future work. Each answer is labeled correct, partial, wrong, hallucinated, or refused. We compare four conditions over the same questions; the three retrieval conditions each insert the top three node bodies. C0 is closed-book generation without retrieval. C1 is the deployed production retriever, a hybrid fusing a lexical and a dense vector index by reciprocal rank fusion. C2 is the proposed ambient injection condition, ranking nodes purely by BGE-M3 dense cosine similarity and delivering them automatically through the hook; the difference from C1 is the ranking engine together with automatic session-time delivery. C3 adds two graph-level signals to C2: F4’s relation triples are attached to each injected node, so the model sees what a node connects to and not only its body, and candidates are reranked by prior reuse. C3 therefore tests whether the structure F4 discovers is worth injecting alongside the text.

3.2. Retrieval quality

Table 1 reports retrieval hit rate and Recall at k . Ambient injection retrieves the answer-bearing node more often than the deployed hybrid baseline at both reported cutoffs. It retrieves the gold node on 44 of the 56 questions against 31 for the baseline, an improvement of 23 percentage points with a 95% paired bootstrap confidence interval of [7, 39]; the table percentages are rounded. The gain is largest on paraphrase questions, where lexical overlap between the question and the source node is low: the hybrid baseline retrieves the gold node in 39% of paraphrase cases, while ambient injection reaches 75%.

3.3. Answer quality

The retrieval improvement is accompanied by a downstream gain in answer correctness. Closed-book generation (C0) is correct on 2% of questions and refuses 72%. The deployed hybrid retriever (C1) raises correct answers to 33%, and ambient injection (C2) to 55%, a 22 percentage-point gain with a 95% paired bootstrap confidence interval of [12, 34]. On the same set, refusals fall from 45% to 29% and hallucinated answers from 12% to 9%; adding graph-level signals (C3) reaches 57%.

When the gold node was retrieved, correct answers reach 64% ($n=44$) against 25% when absent ($n=12$). This is consistent with retrieval being the operative pathway, but the partition was not randomized and the second group is too small for a causal reading; we treat it as exploratory.

3.4. Negative findings

We report two negative findings that bound the contribution. First, graph-level interventions do not help on this corpus: C3 leaves retrieval unchanged and moves correctness by two points, which this sample cannot separate from noise. Reuse-based reranking, predicate-type routing of injected relations, and search-time deduplication each left retrieval hit rate and answer quality unchanged. We treat this as a corpus-specific null rather than evidence that graph interventions cannot help elsewhere: here they did not move retrieval, so they could not move answers.

Second, retrieval is the dominant measured bottleneck, but not the only one. Under an oracle condition that bypasses retrieval by force-injecting the gold node, correct answers on hard compositional questions rise from 7% to 36%. This does not imply that all remaining failures are retrieval failures: even with the gold node supplied, hallucination under the oracle remains at 43%. These residual errors are mostly invention rather than omission, in that the model uses the correct node but adds specifics it does not contain, such as an unsupported numeric factor or a fabricated identifier. Perfect retrieval raises the ceiling without eliminating reasoning error, so independent verification of the grounded answer is future work, not a component evaluated here.

3.5. Domain consistency on real space and nuclear text

To check whether the same behavior appears on real target-domain text, we built a corpus of 123 openly available abstracts (63 space, 60 nuclear) from a public preprint server and posed 18 questions with annotated gold nodes, comparing lexical BM25, dense BGE-M3 (the ambient engine), and their fusion under the same blind protocol. Given the scale we report raw counts and read this as a preliminary consistency check, not a domain benchmark.

The direction of the main finding is reproduced on both retrieval and answer correctness. Dense ambient retrieval finds the gold node for 17 of 18 questions against 16 for BM25, and is graded correct 24 times out of 36 blind gradings against 22 for both BM25 and fusion; since each question is graded twice these are raw counts, not an effective sample of 36. The dense condition also refuses least, while hallucination rates were indistinguishable, so this check speaks to retrieving and answering from the right node, not to hallucination reduction. In a representative paraphrase case the lexical baseline misses the gold node and the model refuses, whereas dense retrieval matches it by meaning and answers correctly. The gap depends on corpus scale: on a smaller 23-node corpus the three methods converge, because every method places the gold node in the top three when retrieval is not the binding constraint.

4. Discussion and limitations

The results support a bounded claim: ambient injection improves retrieval of answer-bearing knowledge over the deployed baseline, most strongly where question and source wording differ, with better grounded answer quality. Graph-level signals add no measured benefit here, and the residual reasoning error makes independent verification the natural next mechanism. These are feasibility measurements from a single generator on general or small domain corpora, not a domain benchmark.

Two limitations bound what these numbers can be attributed to. First, the main comparison contrasts two complete pipelines rather than one variable: C1 and C2 differ in ranking algorithm, embedding model, and candidate-pool size, and only C2 applies the minimum-similarity gate. The gains therefore belong to the pipeline as deployed, not to the trigger, the embedding model, or the gate individually. Second, a single-turn benchmark cannot isolate ambient triggering at all, because any manual-retrieval control would issue the question text as its query and retrieve the same nodes as the hook. The value we ascribe to ambient injection is that it fires when the user states an intent rather than asking a question, surfacing knowledge the user would not have queried for; observing that needs a session-level benchmark, left to future work.

5. Conclusion

We presented an LLM wiki system motivated by high-consequence domains: seven functions over an incremental BGE-M3 embedding layer, with ambient context injection at its center. The deployed ambient pipeline raises the answer-bearing-node hit rate from 55% to 79%, most strongly on paraphrased questions, with higher answer correctness, though this gain belongs to the pipeline as a whole rather than the trigger alone, and retrieval is not the only bottleneck. These results establish implementation feasibility, not domain validation; future work should scale the corpus and add human review.

References

- [1] P. Lewis, E. Perez, A. Piktus, et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, *Advances in Neural Information Processing Systems* 33 (2020) 9459–9474.
- [2] J. Chen, S. Xiao, P. Zhang, et al., BGE M3-Embedding: multi-lingual, multi-functionality, multi-granularity text

embeddings, arXiv:2402.03216 (2024).

- [3] Z. Ji, N. Lee, R. Frieske, et al., Survey of hallucination in natural language generation, *ACM Computing Surveys* 55 (12) (2023) 1–38.
- [4] O. Bensch, L. Bensch, T. Nilsson, et al., Towards a reliable offline personal AI assistant for long duration spaceflight, 75th International Astronautical Congress (IAC), Milan, Italy, 2024.